

Aim

To write a Java program that demonstrates the ListIterator interface methods hasNext(), hasPrevious(), next(), previous(), nextIndex(), previousIndex(), add(), set() and remove() by traversing an ArrayList in both directions and modifying it during traversal.

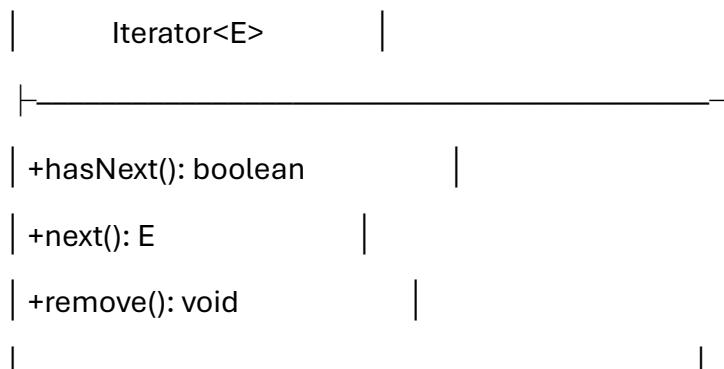
Algorithm

1. Start.
2. Import the java.util package.
3. Create a List<String> object using ArrayList and add the elements Apple, Banana, Cherry and Date. Display the list.
4. Obtain a ListIterator from the list using listIterator().
5. Forward traversal:
 - Repeat while hasNext() returns true.
 - In each iteration, get the index using nextIndex(), then call next() to get the element, and display both.
6. Display nextIndex() and previousIndex() at the end of the list.
7. Backward traversal:
 - Repeat while hasPrevious() returns true.
 - In each iteration, get the index using previousIndex(), then call previous() to get the element, and display both.
8. Display the value of hasPrevious() after the loop. It is false.
9. Obtain a new list iterator and traverse forward. When the element is "Banana", replace it with "Blueberry" using set(). Display the list.
10. Obtain a new list iterator and traverse forward. When the element is "Cherry", delete it using remove(). Display the list.
11. Obtain a new list iterator and traverse forward. When the element is "Apple", insert "Apricot" after it using add(). Display the list.
12. Stop.

UML Diagram

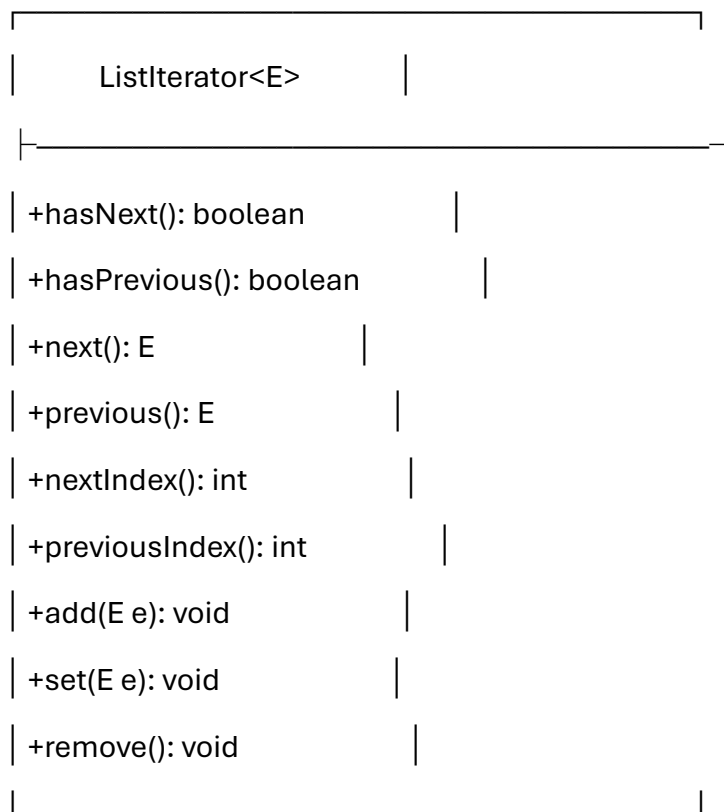
<<interface>>





| (extends)

<<interface>>

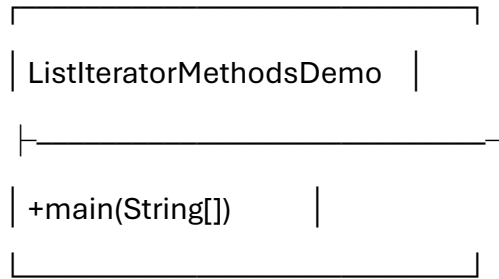


| (implements) | (created by)

ArrayList's / LinkedList's <<interface>>

internal list iterator List<E>

+listIterator(): ListIterator<E>



(uses ListIterator<String>)

Program:

```
package Observations;
```

```
import java.util.*;
```

```
public class ListIteratorInterface {
```

```
    public static void main(String[] args) {
```

```
        List<String> list = new ArrayList<>(  
            Arrays.asList("Apple", "Banana", "Cherry", "Date"));
```

```
        System.out.println("Original list      : " + list);
```

```
        ListIterator<String> li = list.listIterator();
```

```
        // Forward traversal: hasNext(), nextIndex(), next()
```

```
        System.out.println("Forward traversal:");
```

```
        while (li.hasNext()) {
```

```
            int index = li.nextIndex();
```

```
            String fruit = li.next();
```

```
            System.out.println("Index " + index + " : " + fruit);
```

```
        }
```

```
        System.out.println("nextIndex() at end   : " + li.nextIndex());
```

```
        System.out.println("previousIndex() at end : " + li.previousIndex());
```

```
// Backward traversal: hasPrevious(), previousIndex(), previous()

System.out.println("Backward traversal:");

while (li.hasPrevious()) {

    int index = li.previousIndex();

    String fruit = li.previous();

    System.out.println("Index " + index + " : " + fruit);

}

System.out.println("hasPrevious() at start : " + li.hasPrevious());
```

```
// set()

ListIterator<String> li2 = list.listIterator();

while (li2.hasNext()) {

    if (li2.next().equals("Banana")) {

        li2.set("Blueberry");

    }

}

System.out.println("List after set()    : " + list);
```

```
// remove()

ListIterator<String> li3 = list.listIterator();

while (li3.hasNext()) {

    if (li3.next().equals("Cherry")) {

        li3.remove();

    }

}

System.out.println("List after remove() : " + list);
```

```
// add()
```

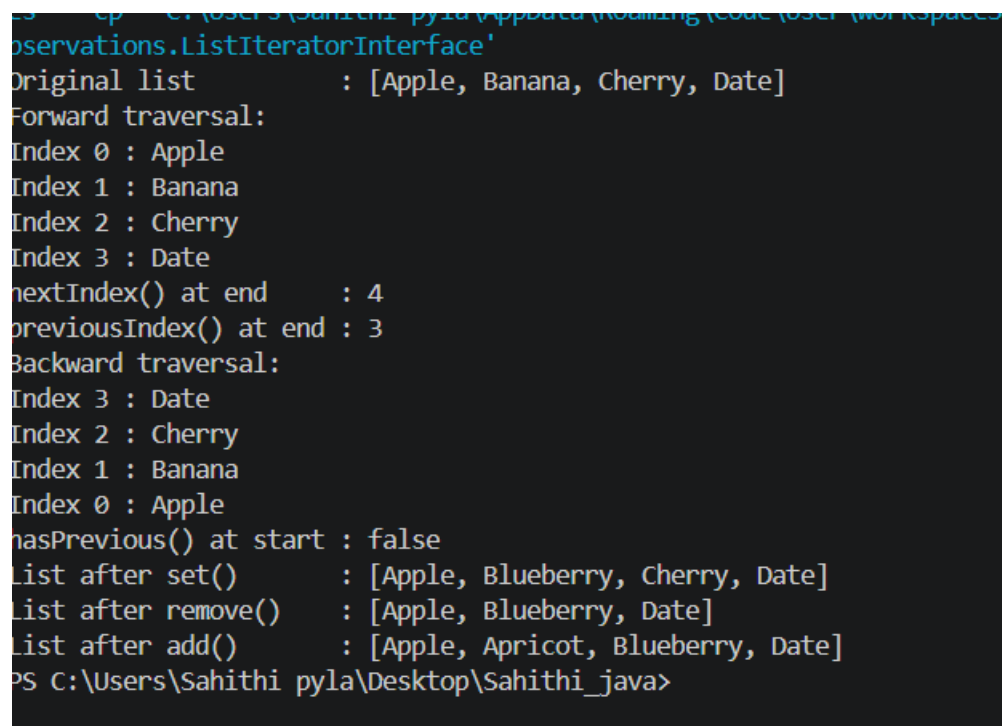
```

ListIterator<String> li4 = list.listIterator();
while (li4.hasNext()) {
    if (li4.next().equals("Apple")) {
        li4.add("Apricot");
    }
}

System.out.println("List after add()    : " + list);
}
}

```

Output:



```

Observations.ListIteratorInterface'
Original list      : [Apple, Banana, Cherry, Date]
Forward traversal:
Index 0 : Apple
Index 1 : Banana
Index 2 : Cherry
Index 3 : Date
nextIndex() at end    : 4
previousIndex() at end : 3
Backward traversal:
Index 3 : Date
Index 2 : Cherry
Index 1 : Banana
Index 0 : Apple
hasPrevious() at start : false
List after set()      : [Apple, Blueberry, Cherry, Date]
List after remove()   : [Apple, Blueberry, Date]
List after add()      : [Apple, Apricot, Blueberry, Date]
PS C:\Users\Sahithi\pyla\Desktop\Sahithi_java>

```

Result:

Thus, the Java program demonstrating the ListIterator interface methods was executed successfully and the output was verified.

- ListIterator works in both directions, using hasNext()/next() going forward and hasPrevious()/previous() going backward.

- `nextIndex()` returns the index of the element `next()` would return, and `previousIndex()` returns the index of the element `previous()` would return.
- `set()` and `remove()` act on the last element returned by `next()` or `previous()`.
- `add()` inserts the new element immediately before the element `next()` would return, so the new element is not visited in the same forward pass.

`set()` and `remove()` throw `IllegalStateException` if `next()` or `previous()` has not been called yet, or if `add()` or `remove()` was called after the last `next()/previous()`. So calling `set()` right after `add()` is illegal.